

LECTURE 14

The Enhanced ER Model

Specialization, generalization, aggregation, and categorization

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- 1 Why plain ER modeling breaks down for domains with natural sub-types
- 2 Specialization (top-down) and generalization (bottom-up) — same diagram, opposite reasoning
- 3 Superclasses, subclasses, and inheritance of attributes AND relationships
- 4 The four disjoint/overlapping × total/partial constraints — and the reasoning behind each choice
- 5 Aggregation: treating a relationship itself as a participant in another relationship
- 6 Composition: aggregation's stronger, ownership-implying cousin
- 7 Categorization (union types): a subclass drawn from several unrelated superclasses

Why Plain ER Isn't Enough

- Plain ER (Lectures 12–13) is entities, relationships, and attributes, all at the same flat level. It breaks down specifically when a domain has natural sub-types sharing a common core but each adding its own extra structure:

Attribute overload

Cram every subtype's attributes into one entity.
Subtype-only columns sit NULL on most rows.

Lost commonality

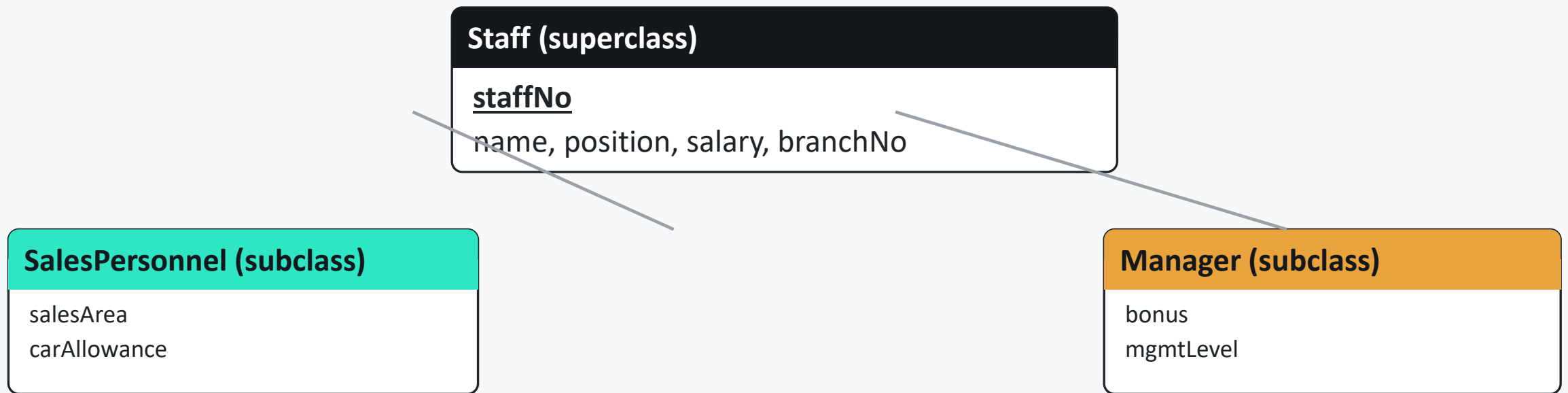
Model SalesPersonnel and Manager as two separate entities. Shared columns are duplicated, and querying all staff needs a manual union.

THE EER MODEL'S SUPERCLASS/SUBCLASS MACHINERY SOLVES BOTH AT ONCE

Shared attributes and relationships live ONCE, on the superclass, and each subclass adds only what's genuinely specific to it. This lecture adds four new modeling concepts on top of Lectures 12–13: specialization, generalization, aggregation, and categorization.

Specialization — Top-Down

- Start from ONE superclass already known to the model (Staff), and identify meaningful, distinct subgroupings based on a distinguishing characteristic (job role).

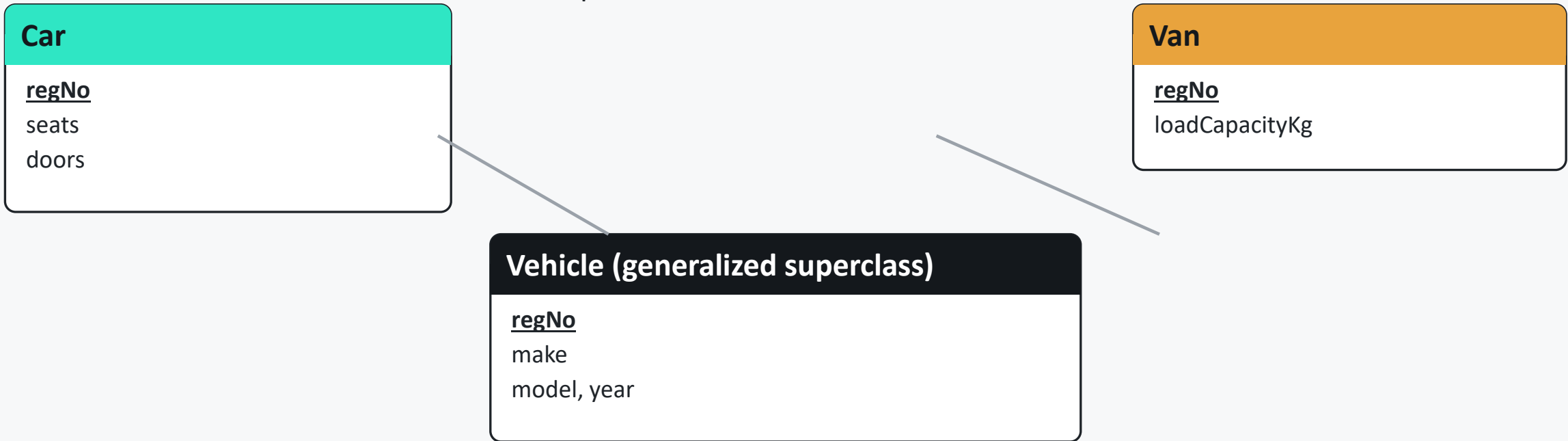


THE DESIGNER'S REASONING

“I already have Staff. Within it, sales personnel and managers need extra, DIFFERENT attributes I have nowhere good to put — let me specialize Staff into subclasses that each carry only their own extra attributes.”

Generalization — Bottom-Up

- Start from several already-known, separately modeled entity types that share a meaningful common core, and factor that shared core out into a new superclass.



THE DESIGNER'S REASONING, RUNNING THE OPPOSITE DIRECTION

“I already separately modeled Car and Van, each with regNo/make/model/year duplicated — let me generalize the shared attributes upward into a new Vehicle superclass, leaving each subclass with only what's genuinely specific to it.”

Superclasses, Subclasses, and Inheritance

- Specialization and generalization produce the IDENTICAL final diagram shape — they differ only in which direction the designer reasoned. Every occurrence of a subclass IS SIMULTANEOUSLY an occurrence of its superclass (every Manager row is also a Staff row) — the same “is-a” relationship object-oriented programming calls inheritance:

Attribute inheritance

A subclass automatically has every attribute its superclass has, without redeclaring them. Manager has staffNo, name, position, salary, branchNo (inherited) plus bonus, mgmtLevel (its own).

Relationship inheritance

A subclass automatically participates in every relationship its superclass participates in — Staff Manages PropertyForRent applies to SalesPersonnel and Manager too, with no separate relationship drawn per subclass.

Subclass-only relationships

A subclass may ALSO have its own additional relationships other subclasses don't — e.g. only Manager participates in Manager Approves Expenditure.

Deciding the Constraints: the Questions to Ask

- Decide each answer from the ACTUAL business rules, never from whichever combination is simpler to draw.

Question 1 — Disjoint or overlapping?

Could one real occurrence be in two subclasses at the same time? If the business would say “this one is both,” it is overlapping. If the roles are exclusive by definition, it is disjoint.

Question 2 — Total or partial?

Must EVERY superclass occurrence fall into some subclass? If the brief lists the roles and nothing else, the list is the evidence that the specialization is total.

WHY THIS MATTERS

A wrong answer is a silent design bug, and it changes which mapping strategy works (Lecture 16).

Constraints on Specialization / Generalization

Disjoint (d)

An occurrence can belong to AT MOST ONE subclass. A Staff member is either SalesPersonnel or Manager, never both.

Overlapping (o)

An occurrence CAN belong to more than one subclass simultaneously. A Person can be both a Student and an Employee at the same time.

Total

Every superclass occurrence MUST belong to at least one subclass — no “plain” unspecialized occurrence exists.

Partial

Some superclass occurrences MAY belong to no subclass at all, remaining a plain, unspecialized instance.

Every specialization in this course is one of exactly four combinations of these two constraints.

The Four Disjoint/Overlapping × Total/Partial Combinations

Disjoint, Total

Vehicle → Car, Van, Truck. Every vehicle is exactly one of the three; none is left unclassified.

Disjoint, Partial

Staff → SalesPersonnel, Manager. Neither overlaps the other, but plenty of staff hold neither role.

Overlapping, Total

Person → Student, Employee. Every person is at least one; some are both.

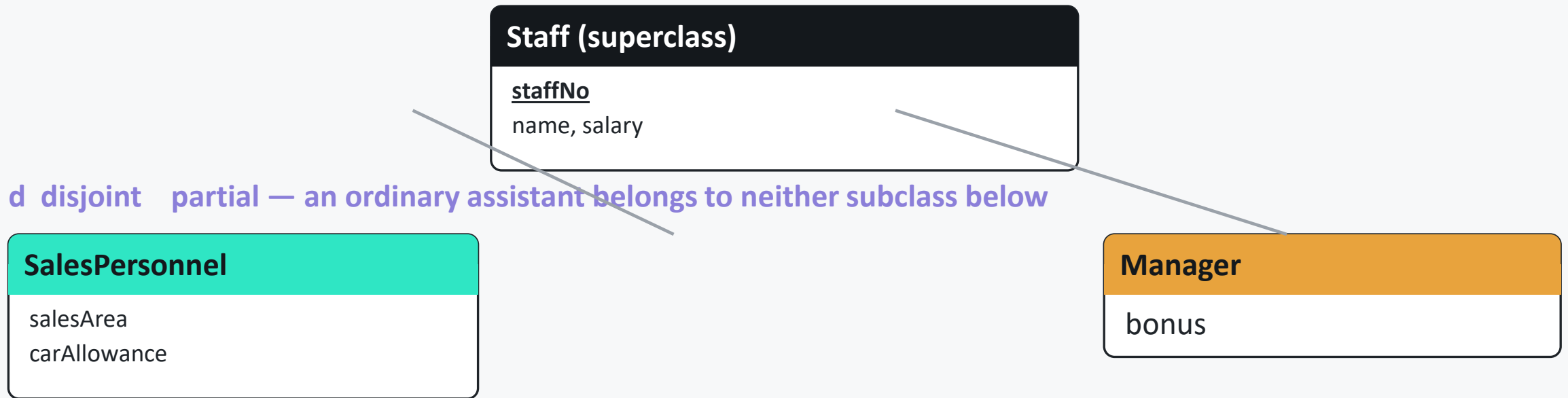
Overlapping, Partial

Staff → CommitteeMember, Trainer. A staff member can hold both extra roles, one, or neither.

A REAL, SILENT DESIGN BUG

Disjoint is wrong if a staff member can be promoted while still holding sales targets: the database silently drops a true role.

Worked Example — Staff Is Disjoint AND Partial

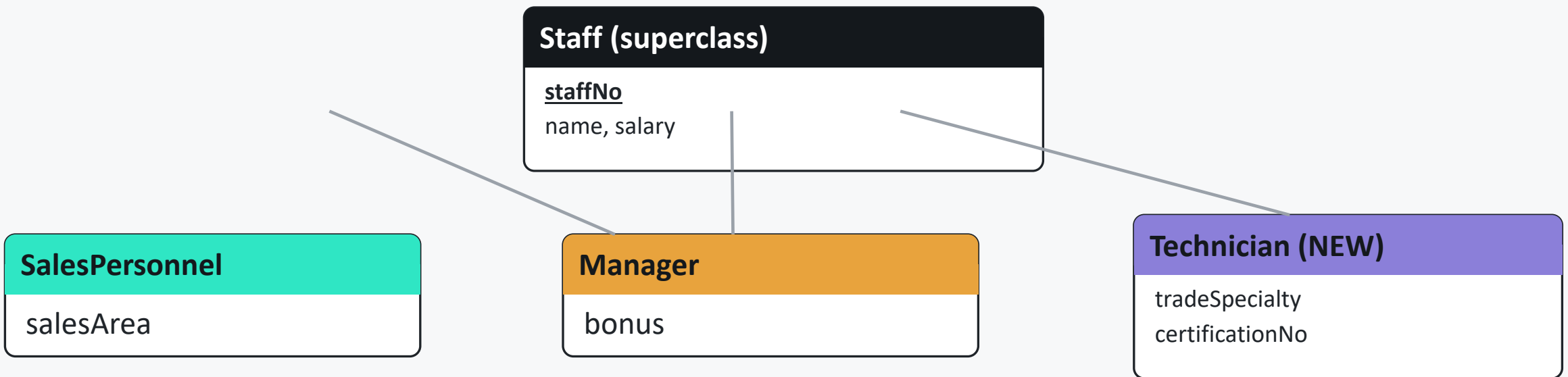


WORKING THROUGH BOTH QUESTIONS

Q1: the roles are mutually exclusive, so disjoint. Q2: a plain assistant belongs to no subclass, so partial. Each answer rests on a checkable fact.

Adding a Third Subclass: Technician

- The agency now also employs maintenance Technicians, with their own tradeSpecialty and certificationNo. Re-running both questions against the GROWN set of roles:



RE-CHECKING BOTH ANSWERS AFTER GROWTH

Q1 stays disjoint and Q2 stays partial: both judge each subclass against the **WHOLE** superclass. Re-check when a hierarchy grows.

Contrast: What If It Were Overlapping?

- Suppose S009 is promoted to Manager but keeps her sales targets and carAllowance.

Under disjoint (current design)

- S009 may appear in one subclass, never both.
- She must lose her sales data or her bonus.
- True, still-current data is silently dropped.

Under overlapping (the fix)

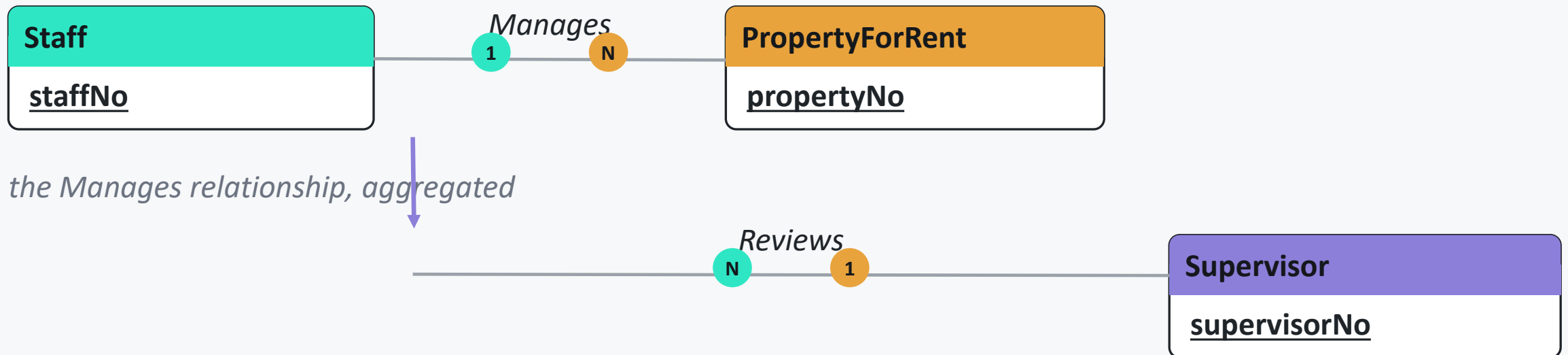
- S009 sits in BOTH subclasses, one shared staffNo.
- Her sales data and her bonus are both kept.
- Queries must handle staff with two roles.

THE GENERAL LESSON

Choose the constraint that a concrete scenario does not break, not the one that is simpler to draw.

Aggregation

- Aggregation models a “has-a” relationship AT THE RELATIONSHIP LEVEL — a relationship between two entity types must itself participate, as a whole, in a relationship with a third entity type.



WHY REVIEWS CANNOT ATTACH TO ONE SIDE

A review is about the PAIRING, not either side alone. Aggregation lets a relationship act as one participant in another.

Aggregation: the Wrong Alternative First

- Try the two “obvious” attachments and see where each breaks:

A: Reviews attaches to Staff

A review cannot say WHICH property it judged. If SG14 manages PG4 and PG16, both reviews look identical.

B: Reviews attaches to property

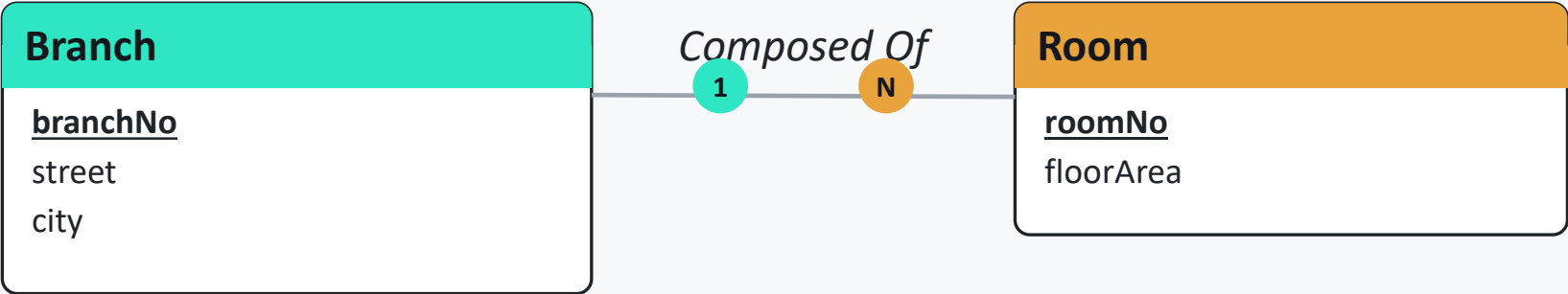
A review cannot say WHOSE management it judged. After a handover from SG14 to SG37, old reviews are ambiguous.

WHY BOTH ATTEMPTS LOSE INFORMATION

Both fail the same way: a review is about the PAIRING, and neither side alone holds it. Aggregation attaches the review to the pairing itself.

Composition

- Composition is a whole-part relationship where the part has NO INDEPENDENT EXISTENCE apart from the whole — and a part typically belongs to exactly one whole at a time.



	Aggregation	Composition
Existence dependency	The part can outlive the whole	The part CANNOT exist without the whole
Sharing	A part can belong to more than one whole	A part belongs to exactly one whole
Deleting the whole	Parts typically left in place / reassigned	Parts deleted along with the whole

Composition: Applying the Decision Criteria

- Branch Composed-Of Room is tested against the same two questions:

Does the part exist apart from the whole?

No: a Room means nothing without its Branch. A property advertisement, by contrast, can be reused elsewhere, so that is aggregation.

Is the part ever shared across wholes?

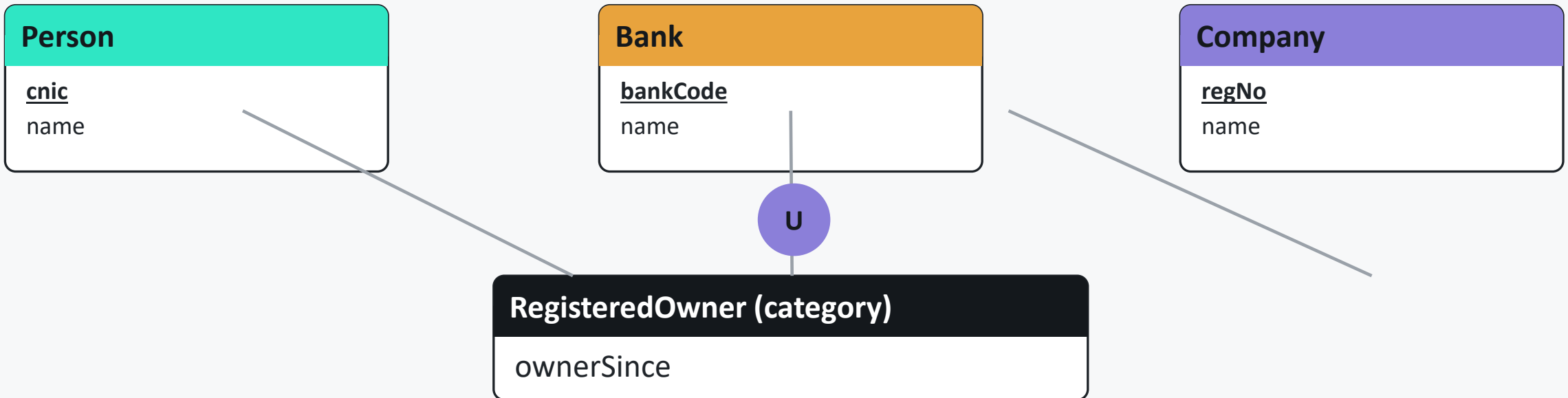
No: a physical Room belongs to exactly one Branch, permanently.

BOTH “NO” ANSWERS MEAN COMPOSITION

Two “no” answers mean composition. It maps to ON DELETE CASCADE in Lecture 16.

Categorization (Union Types)

- A category's subclass represents a subset of the UNION of two or more distinct, otherwise unrelated superclasses — those parents don't share a common superclass of their own.



Categorization: Why Not Generalization?

- The first instinct is a shared “Party” superclass for Person, Bank, and Company. It breaks down:

A forced “Party” superclass

They share little beyond a name, and their keys differ in type (cnic, bankCode, regNo). A shared superclass would invent structure.

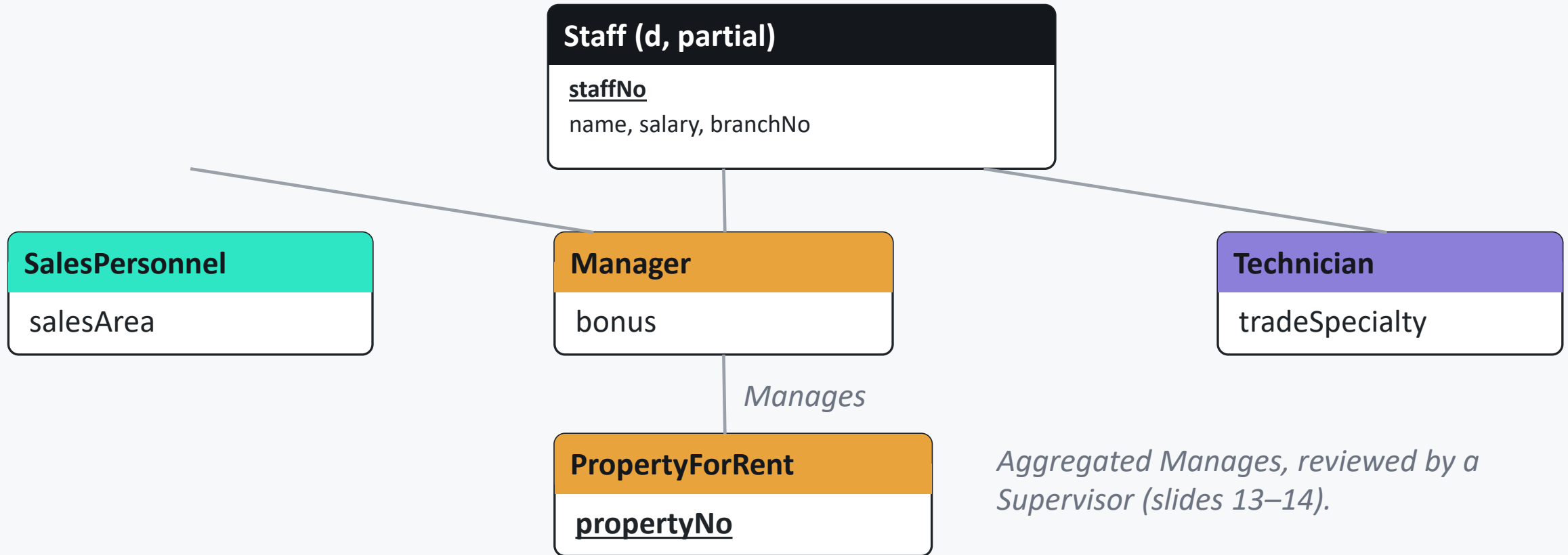
Categorization fits

Each owner type keeps its own identity. RegisteredOwner sits above them only for one relationship: Vehicle's “Registered To.”

DECIDING THE TWO

Generalize when subclasses share attributes the business treats as one fact. Categorize when only one relationship ties them.

Putting It Together: the Full Staff Hierarchy



ONE EXAMPLE, EVERY CONSTRUCT

Specialization (slides 4, 10–11) and aggregation (slides 13–14) run on one example.

KEY TAKEAWAYS

Key Takeaways

- 1 Specialization reasons top-down, generalization bottom-up — both produce the same diagram shape; a subclass inherits every attribute and relationship of its superclass.
- 2 Disjoint/overlapping and total/partial are decided by checking concrete scenarios against the business rules — never by which combination is simpler to draw.
- 3 Aggregation lets a relationship itself act as a single participant in another relationship — needed whenever a fact is genuinely about a pairing, which two “obvious” single-entity attempts can't represent.
- 4 Composition is aggregation plus two “no” answers: no independent existence, no sharing across wholes.
- 5 Categorization fits candidates with no shared attributes to generalize, tied together only by one relationship that needs them all to play the same role.